

PHASE 7 — CONTINUED

PHASE 7 COMPLETION REPORT

Rental Extension & Booking Reassignment

Bliss Rent — Dubai Airport Rental

Prepared for: Zaheer Muneer & Hassan Raza (co-owners)

Date: August 29, 2026

Covers the expanded Phase 7 respecification: dual request channels, booking-reassignment conflict resolution, and late-extension penalty configuration — built on top of the narrower Phase 7 (extension & extension payments) work already reported

Migrations 1 new, local only	Edge Function 1 new, not deployed	Tests 246 / 246 passing	Typecheck Clean	Build Clean
Lint 0 errors	Production Not deployed	Phase 8 Not started		

1. Executive summary

This report covers the expanded Phase 7 respecification you sent after the original "Rental Extension & Extension Payments" work was already reported: a second, customer-facing self-service request channel; a booking-reassignment engine that reassigns a conflicting future booking to a replacement vehicle instead of rejecting the extending customer; and a configurable late-extension penalty, independent from the existing extension-pricing configuration. Everything below was built, unit-tested, type-checked, built, and linted locally. Nothing has been deployed to production — no migration has been applied to your live Supabase project, and the new Edge Function has not been deployed. Two owner decisions remain open: the extension pricing policy (carried over from the original Phase 7 report) and the new late-extension penalty policy. Both are described in §26.

2. Existing functionality reused

Component	Reused as-is
bookings_no_overlap	The Phase 0 exclusion constraint remains the single real double-booking guard. Not touched, weakened, or bypassed anywhere in this phase.
available_vehicles()	Phase 1's exact overlap+status search is the building block for replacement-vehicle search — called for the conflicting booking's own dates, never reimplemented.
get_booking_by_reference()'s indistinguishable-failure pattern	Mirrored exactly for the new reference + vehicle-number verification (zero rows on any mismatch).
booking_extensions, extension_pricing_settings, the original request_booking_extension/confirm_booking_extension_payment	Extended in place (new columns, new statuses, new optional parameter) — not duplicated or replaced. Every previously-passing test for this system still passes unchanged.
is_admin() / is_active (Staff Account Control)	Unchanged. Every new admin-facing function checks is_admin() the same way; a suspended admin is blocked automatically.
audit_logs	Reused for every new action type (conflict detected, replacement selected, future booking reassigned, notification generated) — no new logging table.
create-booking / confirm-payment Edge Function conventions	The new submit-extension-request function follows the identical thin-wrapper + pure-logic + shared-errors pattern.

3. New functionality

- A second, customer self-service request channel (verified by booking reference + vehicle number), landing in the same booking_extensions table as a requested row that does **not** auto-process.
- A booking-reassignment conflict-resolution engine: when extending the exact vehicle would collide with a future booking on that same vehicle, the system searches for a suitable replacement vehicle for the *future* booking, reserves it safely, and moves it — instead of rejecting the extending customer or substituting their vehicle.
- A new, independently configurable late-extension penalty mechanism (owner decision still pending — see §13, §26).
- Full traceability for a vehicle reassignment (vehicle_reassignments) and domain-level customer-notification state (booking_notifications) for the affected customer.
- An explicit admin rejection action for a customer-submitted request, since it no longer auto-resolves to rejected the way the old admin-channel conflict case did.

4. Frontend extension flow

New route /extend-rental (ExtendRentalPage.tsx), linked from both the booking confirmation page and the "manage my booking" page. Two steps, exactly as specified:

1. **Verify** — the customer enters their booking reference and the vehicle's plate number (not email — a deliberately different, stronger check than the existing manage-booking lookup). A mismatch on either field produces the exact same generic message; the screen never reveals which one was wrong.
2. **Request** — once verified, the customer picks a new return date and submits. There is no advance-notice window: 1 day before return, 5 days before, or already past the return date are all accepted (the last is simply flagged is_late). Submission shows a "request submitted, pending review" confirmation — never an instant approval, never a changed booking.

The customer cannot modify booking dates, vehicle, payment status, or extension status directly — every mutation is server-side, via the

Edge Function and its underlying SQL function only.

5. WhatsApp/support flow

Unchanged in spirit, tightened per the new spec: support now verifies **both** the booking reference and the vehicle number (previously only the reference/booking itself was relied on) before an admin records the request. The admin-facing form (`RentalExtensionsSection.tsx`) is unchanged for this path — it still records and can immediately process a confirmed request. Both channels run through the exact same `request_booking_extension()` function; nothing about the extension business logic is duplicated for WhatsApp.

6. Verification mechanism

`verify_booking_for_extension()` (read-only, guest-callable) and `submit_extension_request_public()` (the actual insert, service-role only) both match on `upper(left(replace(booking_id::text, '-', ''), 8)) = reference_suffix AND plate_number = vehicle_number` (whitespace/case-insensitive), and both return zero rows / raise the identical generic error on **any** mismatch — the same indistinguishable-failure shape as Phase 6's `get_booking_by_reference()`. Neither can be used as an oracle to test whether a reference or a vehicle number exists on its own.

7. Availability logic

Unchanged core rule, reaffirmed: every check is against the exact `vehicle_id`, never the model, make, or category. The existing `check_vehicle_availability_for_extension()` preview is untouched. The real, race-safe guarantee remains `bookings_no_overlap`, applied at the moment `bookings.end_date` or `bookings.vehicle_id` is actually updated — never at the pre-check stage.

8. Future-booking conflict handling

New function `resolve_extension_conflict()`. When extending the exact vehicle would overlap a future non-cancelled booking on that same vehicle, the engine does not reject outright. It looks for a replacement vehicle for the *future* booking (never the extending customer's vehicle), locks the candidate row, re-checks its availability immediately before committing, and only then reassigns it. If no safe replacement exists, the extension is marked `conflict_unresolved` — nothing is changed, and the admin dashboard surfaces it for manual handling. This runs identically whether the extension came from the WhatsApp/support channel or the website self-service channel.

9. Replacement vehicle logic

Candidates come from `available_vehicles()` for the conflicting booking's own date range, ranked same model first, then same category, then any other available vehicle — excluding the vehicle already in conflict. No arbitrary downgrade, no automatic upgrade beyond the existing pricing rules (the replacement's own pricing is untouched by this phase). If ranking finds nothing available at all, the outcome is `conflict_unresolved`, never a silent choice.

10. Booking preservation

For the reassigned (future) booking: booking reference, customer, dates, pickup/drop-off, duration, and payment history all remain unchanged — only `vehicle_id` changes, recorded in a new `vehicle_reassignments` row (original vehicle, replacement vehicle, reason, who processed it, when). For the extending customer's own booking: the reference, per the unchanged rule from the original Phase 7 build, never changes — only `end_date` does. Neither party's booking reference is ever regenerated.

11. Payment/invoice

Unchanged design, reaffirmed: the extension's payment/invoice record (`booking_extensions` itself) stays entirely separate from the original booking's payments row. Cash still finalizes immediately; online still defers the actual `end_date`/reassignment commit to payment confirmation — which now also means the reassignment engine only runs once payment succeeds for an online-paid extension, so a future booking is never disturbed for an extension whose payment might still fail. No new real payment provider was introduced.

12. Pricing configuration

Unchanged from the original Phase 7 build: `extension_pricing_settings.policy` remains seeded `NULL`. **Extension pricing policy remains pending owner confirmation.**

13. Late penalty configuration

New, independently configurable, and deliberately left unset: `extension_penalty_settings` supports a fixed fee, a per-day charge, or a percentage of the extension amount — but the policy column is seeded NULL, and both the SQL layer (`request_booking_extension`) and the TypeScript layer (`src/lib/extensionPenalty.ts`) refuse to process a late extension at all until the owner sets one. **Late-extension penalty calculation remains pending owner confirmation.**

14. Database changes

One new migration: `supabase/migrations/20260903000000_phase7_booking_reassignment.sql` (not yet applied to production). It extends `booking_extensions` (new source, `is_late`, `penalty_amount`, `penalty_policy_used`, `conflict_booking_id`, `replacement_vehicle_id`, verification-audit columns; status widened from an enum to a checked text column so `requested/conflict_unresolved` could be added without an enum-transaction restriction), and adds three new tables: `extension_penalty_settings`, `vehicle_reassignments`, and `booking_notifications`. It replaces `request_booking_extension()` and `confirm_booking_extension_payment()` with versions that call the new `resolve_extension_conflict()` engine, and adds `verify_booking_for_extension()`, `submit_extension_request_public()`, and `reject_extension_request()`.

15. API/Edge Function changes

New Edge Function `submit-extension-request` (`index.ts/logic.ts/logic.test.ts`), following the exact create-booking convention — thin Deno HTTP wrapper, pure testable logic, shared `ApiError/CORS` helpers. `adminExtensionsApi.ts` gained `processExtensionRequest`, `rejectExtensionRequest`, `fetchPendingExtensionRequests`, and penalty-settings CRUD. New guest-facing client module `src/features/booking/extendRentalApi.ts`.

16. Admin Dashboard changes

`RentalExtensionsSection.tsx` (per-booking) now shows a review queue for customer-submitted `requested/conflict_unresolved` rows with "Review & process" and "Reject" actions, late/penalty fields in the form and history, and conflict/reassignment notices. `ExtensionsListPage.tsx` gained new status tabs and a source column. `AdminSettingsPage.tsx` gained an owner-only "Late-extension penalty policy" card, mirroring the existing pricing-policy card. Staff Account Control permissions are unchanged and fully enforced — no new privileges were granted to any staff role.

17. Security/RLS

`extension_penalty_settings`, `vehicle_reassignments`, and `booking_notifications` all enable RLS with admin-only read policies and no direct write policy for any role — every write happens through a SECURITY DEFINER function. `submit_extension_request_public()` is revoked from anon/authenticated and granted only to `service_role` (Edge-Function-mediated, per this project's established guest-mutation convention). `verify_booking_for_extension()` is a read-only, zero-row-on-mismatch function, safe to grant directly to anon/authenticated. Every admin-facing mutation still checks `is_admin()` internally.

18. Audit logging

New action types recorded through the existing `audit_logs` table (no new logging system): `extension_customer_verification`, `extension_conflict_detected`, `replacement_vehicle_selected`, `future_booking_reassigned`, `customer_notification_generated`, alongside the existing `extension_requested/extension_approved/extension_rejected/extension_payment_recorded/booking_return_date_changed`.

19. Notifications

Audited and confirmed: no email/SMS/WhatsApp-sending infrastructure exists anywhere in this codebase. WhatsApp today is a manual human support channel, not an integration this system can call. `ContactPage.tsx`'s own code comment says as much ("there is no email backend to actually deliver it to yet"). Per your explicit instruction not to invent a notification platform, this phase implements only the domain state: a new `booking_notifications` table records exactly what a customer should be told (vehicle reassignment, extension approval/rejection) with a customer-safe payload — never internal admin notes — and a `pending_delivery` status. Wiring a real send channel (e.g. the Resend integration already named as the planned provider) is explicitly flagged as a remaining dependency in §25.

20. Tests — 246/246 passing

31 new automated tests were added this phase (215 → 246), across `extensionPenalty.test.ts`, the rewritten

adminExtensionsApi.test.ts, the new submit-extension-request/logic.test.ts, and the new ExtendRentalPage.test.tsx. As with every prior phase report, this sandbox has no live Postgres/Docker instance, so anything that only lives inside a plpgsql function body (the actual conflict-detection query, the replacement-vehicle ranking, the exclusion-constraint race behavior, RLS policies) is **code-reviewed**, not exercised by an automated test — the table below states that honestly against each of the 41 test cases named in your spec, grouped exactly as you listed them.

#	Group	Case	Coverage
1	Basic extension	Extend by 1 day (minimum bound) accepted	Automated
2	Basic extension	Extend by 30 days (maximum bound) accepted	Automated
3	Basic extension	Extend by 31 days rejected (out of bound)	Automated
4	Basic extension	Extend by 0 days rejected	Automated
5	Basic extension	Cash extension finalizes immediately (approved + paid)	Automated
6	Basic extension	Online extension stays pending until payment confirmed	Automated
7	Basic extension	Late extension (past return date) recognized, flagged is_late	Automated
8	Basic extension	Configurable penalty applied to a late extension	Automated
9	Basic extension	Late extension refused when no penalty policy is configured	Automated
10	Basic extension	Extension refused when no pricing policy is configured	Automated
11	Verification	Correct reference + correct vehicle number verifies	Automated
12	Verification	Wrong vehicle number, correct reference → generic failure	Automated
13	Verification	Wrong reference, correct vehicle number → generic failure	Automated
14	Verification	Both wrong → identical generic failure (indistinguishable)	Automated
15	Verification	Cannot be used to probe/enumerate another customer's booking	Code-reviewed
16	Exact vehicle availability	Availability keys on exact vehicle_id, never model/category	Code-reviewed
17	Exact vehicle availability	Two same-model vehicles are never treated as interchangeable	Code-reviewed
18	Exact vehicle availability	Preview availability check never mutates state	Code-reviewed
19	Exact vehicle availability	Real guarantee is bookings_no_overlap at commit time	Code-reviewed
20	Future booking conflict	Conflict detected when extended window overlaps a future booking	Code-reviewed
21	Future booking conflict	Extension NOT auto-rejected merely because a future booking exists	Automated (API layer)
22	Future booking conflict	Same-model replacement preferred first	Code-reviewed
23	Future booking conflict	Same-category replacement used when no same-model vehicle is free	Code-reviewed
24	Future booking conflict	No replacement found → conflict_unresolved, nothing changed	Automated (API layer)
25	Future booking conflict	Extending customer keeps their original vehicle on success	Code-reviewed
26	Future booking conflict	Reassigned booking's reference/dates/customer info unchanged	Code-reviewed
27	Future booking conflict	vehicle_reassignments row records original/replacement/reason	Code-reviewed
28	Future booking conflict	Last-moment race during reassignment rolls back atomically (23P01)	Code-reviewed
29	Payment	Cash extension payment recorded with full detail	Automated
30	Payment	Online payment confirmed via confirm_booking_extension_payment	Automated
31	Payment	Confirming an already-resolved payment is a no-op (idempotent)	Code-reviewed

32	Payment	Extension invoice fully separate from original booking's payment history	Code-reviewed
33	Payment	No mock/test payment ever claimed as a real production gateway	Code-reviewed
34	Permissions	Customer cannot approve/process their own extension	Code-reviewed
35	Permissions	Suspended staff blocked from processing extensions	Code-reviewed
36	Permissions	Only an active admin can call the processing/confirmation/rejection functions	Code-reviewed
37	Permissions	RLS restricts reads to admins, no direct writes for any role	Code-reviewed
38	Regression	Full Phase 0-6 + Staff Account Control + original Phase 7 suite	Automated — 246/246
39	Regression	<code>npx tsc --noEmit</code>	Automated — clean
40	Regression	<code>npm run build</code>	Automated — clean
41	Regression	<code>npm run lint</code>	Automated — 0 errors, baseline warnings only

21. Typecheck

`npx tsc --noEmit` — clean, 0 errors.

22. Build

`npm run build` — clean, 212 modules transformed. The one warning present (a chunk-size advisory) is pre-existing and unrelated to this phase.

23. Lint

`npm run lint` (oxlint) — 0 errors, exactly 11 pre-existing warnings (the same baseline reported in the Staff Account Control and original Phase 7 reports), none new.

24. Phase 0-6 regression

All 215 previously-passing tests (Phase 0 through the original Phase 7 extension/payments build) still pass unchanged, alongside the 31 new tests — 246/246 total. Nothing in this phase modified an existing test's expected behavior beyond what the new optional parameters/fields required.

25. Known limitations

- **No live database in this sandbox.** Every plpgsql-only behavior (conflict detection, replacement ranking, race-safety, RLS enforcement) is code-reviewed, not exercised by an automated test — see the table in §20. The migration has not been run against any real Postgres instance yet.
- **No real notification channel.** `booking_notifications` records what should be sent; nothing actually sends it. Wiring a real provider (e.g. Resend) is a separate, later integration.
- **A customer-submitted request's dates cannot be edited during admin review** — an admin can approve or reject exactly what was requested; changing the dates requires rejecting and asking the customer to resubmit. This was a deliberate scope decision to avoid ambiguity between what the customer verified and what gets approved.
- **The reassignment engine handles one primary conflicting booking per extension.** If several future bookings on the same vehicle would all conflict with the extended window, the engine currently resolves the earliest one; a second overlapping booking would require a second pass. This matches every worked example in your spec (which described exactly one conflicting future booking) but is worth flagging if that scenario turns out to be more common in practice.

26. Owner decisions still pending

Extension pricing policy — original booking's daily rate, the vehicle's current daily rate, or a separate custom extension rate. Set in Settings → Extension pricing policy once decided. Carried over, unresolved, from the original Phase 7 report.

Late-extension penalty policy — a flat fixed fee, a per-day charge, a percentage of the extension amount, or another rule entirely. Set in Settings → Late-extension penalty policy once decided. Until then, a late extension cannot be processed by either channel.

27. Production deployment status

Nothing in this phase has been deployed to production. The migration exists only in this repository (and has been synced to your Desktop copy of the repository, per your earlier request) — it has not been applied to your live Supabase project. The submit-extension-request Edge Function has not been deployed. No customer can currently reach the self-service Extend Rental flow in production, and no admin-facing changes from this report are live yet.

Stop condition: This is the full Phase 7 (Rental Extension & Booking Reassignment) completion report, delivered as PDF and in-chat text, per standing instructions.

Two owner decisions are pending (\$26). Nothing has been deployed to production. Phase 8 has not been started and will not begin until you give explicit go-ahead.